Module 6 — Error Handling, Validation, and Robustness in AI APIs

Learning Objectives

By the end of this module, you will be able to:

- **Predict and intercept** common failure points in AI inference pipelines.
- **Implement manual validation checks** for feature ranges and model constraints.
- **Utilize `HTTPException`** to provide clear, actionable error messages to API consumers.
- **Select appropriate HTTP status codes** specifically relevant to machine learning workflows.
- **Debug production crashes** using standardized error responses and logging.

Why AI Systems Fail in Production

In a notebook, a failure is just a red cell that you fix and re-run. In production, a failure can be **silent** or **catastrophic**. A silent failure occurs when the API returns a "200 OK" status but contains a null prediction or a garbage value because the model encountered an unexpected input. This is dangerous because downstream systems (like an automated trading bot or a medical diagnostic tool) will treat the "garbage" as a valid result.

Robustness means your API is designed to fail **loudly and clearly** only when it must, and it protects the model from data that would cause a crash.

Failure Scenarios and Error Types

1. Input Validation Errors (Automatic)

These occur when the data format is wrong (e.g., sending a string like `"high"` to a model that requires a numeric `risk_score`).

- **Why this exists:** To ensure the mathematical integrity of the model's inputs.
- **Where it is used:** Every time a client sends a JSON request body.
- **What breaks:** Without this, the Python interpreter will raise a `TypeError` deep inside your ML library, causing an "Internal Server Error".

2. Model Inference Errors (Manual)

These occur when the data format is correct, but the **content** is invalid (e.g., a "House Age" of -500 years).

- **Why this exists:** Models are often "black boxes" that will try to calculate a result for any numbers given, even if those numbers are physically impossible.
- **Where it is used:** Defensive checks performed *after* validation but *before* calling `model.predict()`.
- **What breaks:** The model provides a nonsensical prediction that can lead to bad business decisions.

3. Preprocessing Mismatches

If your model was trained on data scaled between 0 and 1, but the API receives raw values like 100 or 5000, the prediction will be wrong.

- **Why this exists:** To bridge the gap between "Research" and "Production" data distributions.
- **Where it is used:** Inside the path operation function before the inference step.
- **What breaks:** The API appears to work perfectly but produces highly inaccurate results, making the AI system useless.

Core Concepts for Robustness

1. FastAPI `HTTPException`

This is a specialized Python exception that you can "raise" to immediately stop the code execution and send a specific error to the client.

- **Why this exists:** To provide a clean way to exit a function when an edge case is hit.
- **In AI Systems:** Used to stop inference if feature counts are wrong or if a required model file is missing.
- **If missing:** The server will crash with a full stack trace, which is a security risk and hard for clients to read.

2. AI-Relevant HTTP Status Codes

You don't need to know all 100+ codes. For an AI Engineer, these are the essentials:

- **400 Bad Request:** Use this for **Preprocessing Mismatches** or out-of-range values.
- **404 Not Found:** Use this if the user requests a specific model version or a data ID that doesn't exist in your system.
- **422 Unprocessable Entity:** This is sent **automatically** by FastAPI/Pydantic when the input doesn't match your schema (e.g., missing features).

- **500 Internal Server Error:** This means your code crashed. In a robust system, you want to catch these and turn them into 400s whenever possible.

Code Examples with Line-by-Line Explanations

Example 1: Defensive Check for Feature Ranges

This code ensures a model only receives valid "Probability" inputs (0.0 to 1.0).

```
from fastapi import FastAPI, HTTPException, status #
from pydantic import BaseModel

app = FastAPI()

class InferenceInput(BaseModel):
    feature_a: float

@app.post("/predict")
def predict(data: InferenceInput):
    # 1. Defensive Check:
    if data.feature_a < 0 or data.feature_a > 1.0:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST, #
            detail="Feature A must be between 0 and 1.0" #
        )

    # 2. Simulated Model Inference:
    prediction = data.feature_a * 0.5
    return {"result": prediction}
```

Line-by-Line Explanation:

- **`raise HTTPException(...)`**: This immediately kills the current request. No further code (like the model prediction) is run.
- **`status.HTTP_400_BAD_REQUEST`**: Using the `status` helper ensures you don't make typos with numbers.
- **`detail="..."`**: This message is sent to the client. It helps the user understand *why* their request was rejected without needing to call the AI Engineer.

Example 2: Handling Model Failure Gracefully

Sometimes a model fails for unknown reasons. We can use a `try-except` block to prevent a 500 error.

```
@app.post("/safe-predict")
def safe_predict(data: InferenceInput):
    try:
        # Simulate a model crash (e.g., Matrix Mismatch)
        result = model.predict(data.feature_a)
```

```
        return {"prediction": result}
    except Exception as e:
        # Fallback response
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail=f"Model Inference Failed: {str(e)}"
        )
```

Line-by-Line Explanation:

- **`try:`**: We attempt the inference logic.
- **`except Exception as e:`**: If the model library (like PyTorch or Sklearn) crashes, this block catches it.
- **`detail=f"...{str(e)}"`**: This sends the specific error message back to the client, making it 100% easier to debug "Wrong Predictions" or "Model not loading".

Debugging and Robustness Strategies

1. **Use the Swagger UI (/docs):** Always test your error cases here. Try sending letters instead of numbers to see the automatic **422 error** in action.
2. **Breakpoints in VS Code:** If a model is giving wrong predictions, set a breakpoint *inside* the `try` block. You can inspect the "Local Variables" to see exactly what features reached the model after preprocessing.
3. **Strict Schemas:** Never use a generic `dict` for inputs. Use **Pydantic Models** to force the API to handle the "Wrong feature count" problem automatically.

Common Robustness Mistakes

- **Returning "None":** Returning `{"result": None}` when something goes wrong. The client gets a 200 OK, which might break their code. **Always raise an exception instead.**
- **Leaking Secrets in Errors:** Including full file paths or database credentials in the `HTTPException` detail. Only include messages related to the model or data.
- **Ignoring 422s:** AI engineers often try to write manual `if` statements to check if a value is an integer. Let FastAPI do this for you using **Type Hints**.

Assignment: The "Safe Predictor"

1. Create a Pydantic model for a "Temperature Sensor" that accepts a `float` value called `celsius`.
2. Build a POST `/analyze` endpoint.

3. Add a defensive check: If the temperature is below -273.15 (Absolute Zero), raise a **400 Bad Request** with the message `"Physics violation: Temperature too low"`.
4. If the temperature is above 1000, raise a **400 Bad Request** with the message `"Sensor limit exceeded"`.
5. Otherwise, return `{"status": "normal"}`.

Mini Project: Fault-Tolerant Inference API

Build a "Reliable Classifier" service:

- Define a schema for a 5-item feature vector (List of floats).
- On startup, try to load a "weights" dictionary. If it fails, log a message.
- In the `/predict` endpoint, use a `try-except` block to wrap the logic.
- If the input list does not contain exactly 5 items, raise an **HTTP 400 error**.
- If any mathematical error occurs during the "inference" (simulated multiplication), return an **HTTP 500 error** with the specific technical reason.

Production-Readiness Checklist

- [ ] Are all inputs validated by a **Pydantic Schema**?
- [ ] Does every endpoint have an **explicit Response Model** to prevent leaking raw objects?
- [ ] Are **Defensive Checks** implemented for impossible data values?
- [ ] Do all errors use the **`HTTPException`** class with a clear `detail`?
- [ ] Have you tested the API using the **`/docs`** page with intentionally bad data?
- [ ] If the model crashes, does the API return a **500 error** instead of just timing out?